

Salso e Greedy Search: algoritmi a confronto per l'ottimizzazione bayesiana del clustering dei dati attraverso processo decisionale

Tommaso Menghini matricola 864946

Tommaso Pozzi matricola 864654

22 Novembre 2024

Abstract

L'analisi dei dati tramite un modello mistura di processo di Dirichlet (DPMM) consente di esplorare la distribuzione a posteriori delle partizioni latenti, un aspetto cruciale per i problemi di clustering. Tuttavia, sintetizzare questa distribuzione in una partizione rappresentativa pone una sfida complessa. Questo lavoro esamina l'uso di funzioni di perdita, come *Binder* e *Variation of Information*, per identificare una stima puntuale della partizione ottima. Vengono confrontati due algoritmi, Greedy Search e Salso, che sfruttano campionamenti MCMC per ottenere la stima. Inoltre, si approfondiscono metodi per quantificare l'incertezza delle stime, come le *credible balls*. Il confronto teorico e pratico tra questi approcci mira a valutare efficacia ed efficienza nella sintesi della distribuzione a posteriori, con particolare attenzione alla teoria del reticolo delle partizioni e agli strumenti per la valutazione dell'incertezza. Con uno studio simulativo si arriva alla conclusione che il metodo migliore è la combinazione tra l'algoritmo Salso e la funzione di perdita *VI*. Si è successivamente svolta un'applicazione ad un dataset reale che coinvolge alcune statistiche socio-economiche di alcune nazioni con l'obiettivo di proporre un clustering di quest'ultime.

1 Introduzione al problema del clustering tramite teoria decisionale

Il clustering è una disciplina di statistica e machine learning ampiamente studiata e capace di evidenziare particolari pattern nascosti in dati complessi difficilmente osservabili dall'occhio umano. I più classici algoritmi di clustering, come k-means o metodi gerarchici, sono spesso utilizzati in contesti dove non si hanno particolari conoscenze preliminari sulla distribuzione dei dati. Sebbene questi algoritmi siano ottimi al fine di clusterizzare i dati, essi presentano svariate limitazioni come particolare sensibilità a restare intrappolati in minimi locali o la richiesta della specifica di un determinato numero k di cluster a priori. Un'alternativa interessante per l'analisi dei cluster risiede nei modelli mistura. Questi permettono di sfruttare una particolare distribuzione di probabilità con la quale si pensa siano distribuiti i vari gruppi all'interno dei dati al fine di riconoscere pattern il più omogenei possibile al loro interno. In questo progetto sono stati analizzati metodi di clustering bayesiani non parametrici, utilizzando i risultati ottenuti da Wade and Ghahramani (2018). In particolare, i metodi bayesiani per il clustering permettono di sfruttare conoscenze a priori, cioè informazioni non direttamente contenute nei dati, riguardo alla struttura dei pattern. Un'altra peculiarità dell'approccio Bayesiano è la capacità di quantificare l'incertezza delle stime ottenute. In particolare, verrà studiata la tecnica delle *credible balls*, uno strumento efficace per visualizzare l'instabilità delle stime. Tradizionalmente l'incertezza viene osservata graficamente tramite la *heat map* della posterior similarity matrix. Tuttavia, questo metodo non fornisce una precisa quantificazione dell'incertezza a differenza delle palle di credibilità. I modelli mistura non parametrici assumono un numero di componenti teoricamente infinito, in contraddizione ai modelli parametrici che prevedono un numero finito di raggruppamenti. Questa caratteristica rende i modelli non parametrici flessibili, poiché permettono di considerare un numero di cluster crescente all'aumentare della dimensione del campione. La statistica bayesiana non parametrica consente di co-

struire ed interpretare una distribuzione a posteriori sullo spazio delle partizioni, riflettendo le credenze a priori e l'incertezza derivante dal campione.

Un problema cruciale è rappresentato dal come sintetizzare la distribuzione a posteriori delle partizioni. In Wade and Ghahramani (2018) e in Dahl et al. (2022) questo tema viene approfondito, riflettendo su una domanda principale: quale funzione di perdita può essere considerata ottima per sintetizzare la distribuzione a posteriori e ottenere una stima puntuale dello spazio delle partizioni (partizione ottima $\mathbf{c}^*$)?

Nelle tipiche analisi bayesiane parametriche, la distribuzione a posteriori di particolari parametri viene sintetizzata utilizzando la media a posteriori, la mediana o la moda, quantificando l'incertezza di tali stime attraverso intervalli di credibilità o HPD (High Posterior Density). In questo progetto si sono studiati diversi approcci decisionali per la sintetizzazione della distribuzione a posteriori; implementando diverse funzioni di perdita e algoritmi differenti per riuscire ad ottenere la partizione ottima dei dati. I metodi proposti sono basati sull'analisi della matrice di similarità a posteriori, nota come *posterior similarity matrix*. Le funzioni di perdita proposte e confrontate in questo progetto sono: *Binder Loss* e la sua generalizzata (Sezione 2.1) e la *Variation of Information loss* e le sue varianti (Sezione 2.2). Entrambe queste metriche possiedono proprietà che sono ben allineate con la teoria del reticolo delle partizioni, spiegata in dettaglio nella Sezione 3.0.1. Le due funzioni di perdita verranno confrontate in modo teorico e in maniera pratica attraverso un esempio di simulazione dove ne verrà analizzata l'efficacia e l'efficienza. In Wade and Ghahramani (2018) viene inoltre proposto un algoritmo *greedy* per calcolare la partizione ottima, approfondito nel dettaglio nella Sezione 3.1. A questo si affianca un algoritmo più recente, introdotto in Dahl et al. (2022), che rappresenta un'alternativa più adatta a dati complessi e sarà discusso nella Sezione 3.2 dedicata all'algoritmo SALSO.

1.1 Modello mistura del processo di Dirichlet

Gli algoritmi proposti in questo progetto sono stati implementati per modelli mistura bayesiani non parametrici; in particolare sviluppati per i modelli mistura di processo di Dirichlet (DPMM), in Equazione 1,

$$f(x|P) = \int_{\Theta} K(x|\theta) d\tilde{P}(\theta), \quad (1)$$

dove $K(x|\theta)$ è una specifica distribuzione Kernel sullo spazio campionario con parametro di mixing pari a θ e $\tilde{P}$ è una misura di probabilità aleatoria su Θ , comunemente sostituita dal Processo di Dirichlet, mostrato in Equazione 2:

$$\tilde{P}(\theta) = \sum_{j=1}^{\infty} p_j \delta_{\theta_j}(\theta), \quad (2)$$

dove $\theta \subseteq \Theta$ e dove si assume che la successione dei pesi p_j e la successione dei salti θ_j siano indipendenti e i θ_j siano indipendenti ed identicamente distribuiti da una misura base P_0 (definizione stick-breaking del Processo di Dirichlet (Sethuraman, 1994)). Ciò permette di modellare l' Equazione 1 come un modello mistura infinito:

$$\tilde{f}(x|\tilde{P}) = \sum_{j=1}^{\infty} p_j K(x|\theta_j). \quad (3)$$

Dato che $\tilde{P}$ è discreto quasi certamente, il modello induce ad una partizione latente $\mathbf{c}$ dei dati dove due punti appartengono allo stesso cluster se e solo se sono stati generati dalla stessa componente della mistura. La partizione può essere rappresentata come $\mathbf{c} = (C_1, C_2, \dots, C_{k_N})$, dove C_j contiene gli indici dei punti nel j^{th} cluster e k_N è il numero dei cluster ottenuti dal campione di ampiezza N . La differenza chiave con i classici modelli mistura è che il numero delle componenti è assunto infinito; questo permette ai dati di determinare il numero k_N di cluster presenti internamente.

Definito $\mathbf{x}_j = \{x_n\}_{n \in C_j}$, posso scrivere la verosimiglianza marginale dei dati $x_{1:N}$, data la partizione, come:

$$f(x_{1:N}|\mathbf{c}) = \prod_{j=1}^{k_N} m(\mathbf{x}_j) = \prod_{j=1}^{k_N} \int \prod_{n \in C_j} K(x_n|\theta) dP_0(\theta). \quad (4)$$

E' facilmente ricavabile la distribuzione a posteriori delle partizioni, che riflette le nostre credenze (attraverso la prior) e l'incertezza nei cluster esplicitata attraverso i dati. In Equazione 5 è rappresentata la distribuzione a posteriori delle partizioni $\mathbf{c}$.

$$p(\mathbf{c}|x_{1:N}) \propto p(\mathbf{c}) \prod_{j=1}^{k_N} m(x_j), \quad (5)$$

dove $p(c)$ è la distribuzione a priori. Per esempio, una prior Dirichlet Process per $\tilde{P}$, proposta da Ferguson (1973), con parametro di concentrazione α corrisponde a:

$$p(\mathbf{c}) = \frac{\Gamma(\alpha)}{\Gamma(\alpha + N)} \alpha^{k_N} \prod_{j=1}^{k_N} \Gamma(n_j), \quad (6)$$

dove $n_j = |C_j|$ ossia il numero di osservazioni appartenenti al cluster j .

In generale la verosimiglianza marginale dei dati, date le partizioni, non è calcolabile in forma chiusa. Per questo motivo durante questo studio sono state utilizzate tecniche di simulazione Monte Carlo (MCMC). Nonostante la generazione di un numero elevato di partizioni durante la simulazione Monte Carlo, non è possibile osservare tutte le possibili partizioni dei dati dato che ci sono

$$S_{N,k} = \frac{1}{k!} \sum_{j=0}^k (-1)^j \binom{k}{j} (k-j)^N \quad (7)$$

strade per partizionare gli N punti nei k cluster e un totale di

$$B_N = \sum_{k=1}^N S_{N,k} \quad (8)$$

possibili partizioni.

Chiaramente descrivere tutte le possibili partizioni della posterior attraverso le simulazioni MCMC è impossibile (Basti pensare che per $N = 10$, $B_{20} = 51.724.158.235.372$). L'obiettivo è quindi quello di sviluppare delle strategie per sintetizzare e caratterizzare la distribuzione a posteriori per riuscire ad ottenere una stima ottima dei cluster che risiedono nei dati considerando anche le partizioni che non compaiono nelle simulazioni Monte Carlo.

2 Funzione di Perdita

Lo scopo è quello di trovare una stima puntuale del clustering che sia rappresentativa della distribuzione a posteriori, cioè che permetta di capirne il comportamento. Dalla teoria delle decisioni è noto che specificando una funzione di perdita $L(\mathbf{c}, \hat{\mathbf{c}})$ è possibile ottenere una sintesi della distribuzione a posteriori attraverso una stima puntuale, calcolando la perdita di stimare $\mathbf{c}$, vera partizione dei dati, con $\hat{\mathbf{c}}$ partizione stimata. Trattandosi di un problema di clustering non si è a conoscenza dei veri raggruppamenti nei dati, è quindi prassi calcolare la media a posteriori di tutte le possibili partizioni pesate per la loro probabilità a posteriori. La stima puntuale $\mathbf{c}^*$ corrisponde a quella partizione che minimizza la perdita attesa a posteriori.

$$\mathbf{c}^* = \underset{\hat{\mathbf{c}}}{\operatorname{argmin}} \mathbb{E}[L(\mathbf{c}, \hat{\mathbf{c}})|x_{1:N}] = \underset{\hat{\mathbf{c}}}{\operatorname{argmin}} \sum_{\mathbf{c}} L(\mathbf{c}, \hat{\mathbf{c}}) p(\mathbf{c}|x_{1:N}).$$

Una scelta banale è la funzione di perdita 0-1, definita come $L(\mathbf{c}, \hat{\mathbf{c}}) = \mathbf{1}(\mathbf{c} \neq \hat{\mathbf{c}})$ che assume valore 0 se la stima della partizione è uguale a quella vera, 1 altrimenti. Si dimostra che sotto la perdita 0-1 la stima puntuale ottima è la moda a posteriori. Nonostante ciò questa funzione di perdita possiede un limite

non indifferente, in quanto non tiene in considerazione della 'somiglianza' fra due partizioni: in effetti, se una partizione differisce dalla partizione vera solo per una singola osservazione, viene penalizzata allo stesso modo di una partizione che differisce per l'allocazione di più osservazioni. Questo implica che la penalizzazione non tiene conto della quantità di errore, ma considera ogni differenza tra le partizioni in modo uguale, indipendentemente dal numero di osservazioni coinvolte. Detto questo, costruire una funzione di perdita più generale non è intuitivo perché deve soddisfare certi principi come l'invarianza rispetto alla permutazione degli indici dei dati e rispetto alla permutazione delle etichette dei cluster. In questo studio ci si è concentrati su due funzioni di perdita in particolare la *Binder* (Binder (1978)) e la *Variation of Information* denotata come VI (Meilă (2007)).

2.1 Binder e Binder generalizzata

La funzione perdita *Binder* (Binder, 1978) è una funzione quadratica delle frequenze n_{ij} pari alla cardinalità dell'intersezione tra C_i , l'insieme degli indici dei punti nel cluster i sotto la partizione $\mathbf{c}$ e $\hat{C}_j$, l'insieme degli indici dei punti nel cluster j sotto $\hat{\mathbf{c}}$, per $i = 1, \dots, k_N$ e $j = 1, \dots, \hat{k}_N$. Con la notazione k_N e $\hat{k}_N$ si rappresenta il numero di cluster rispettivamente in $\mathbf{c}$ e $\hat{\mathbf{c}}$. Per ogni possibile coppia di osservazioni, la funzione di perdita *Binder* penalizza due tipologie di errori:

1. L'assegnazione di due osservazioni a due cluster diversi quando avrebbero dovuto far parte dello stesso;
2. L'assegnazione di due osservazioni allo stesso cluster quando avrebbero dovuto far parte di due diversi;

$$B(\mathbf{c}, \hat{\mathbf{c}}) = \sum_{n < n'} (a \cdot \mathbf{1}\{c_n = c_{n'}\} \mathbf{1}\{\hat{c}_n \neq \hat{c}_{n'}\} + b \cdot \mathbf{1}\{c_n \neq c_{n'}\} \mathbf{1}\{\hat{c}_n = \hat{c}_{n'}\}),$$

con $a > 0$ e $b > 0$ che sono il costo per unità di compiere un errore di miss classificazione. In particolare a si riferisce alla tipologia di errore al punto 1, mentre b si riferisce alla tipologia al punto 2.

In Wade and Ghahramani (2018) viene introdotta una versione invariante rispetto ad N , dimensione campionaria, sotto l'assunzione che $a = b = 1$,

$$B(\mathbf{c}, \hat{\mathbf{c}}) = \sum_{i=1}^{k_N} \left(\frac{n_{i+}}{N} \right)^2 + \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{+j}}{N} \right)^2 - 2 \sum_{i=1}^{k_N} \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{ij}}{N} \right)^2, \quad (9)$$

con $n_{i+} = \sum_i n_{ij}$ e $n_{+j} = \sum_j n_{ij}$. Questa espressione è invariante rispetto a N in quanto dipende dalla dimensione del campione solo tramite le proporzioni n_{ij}/N . Questa funzione di perdita è conveniente perché sfrutta i risultati successivamente presentati. In Dahl (2006) viene proposto il criterio dei minimi quadrati per cercare la partizione che minimizza $f(\hat{\mathbf{c}}) = \sum_{n=1}^N \sum_{n'=1}^N (A(\hat{\mathbf{c}})_{nn'} - \hat{\pi}_{nn'})^2$, con $A(\hat{\mathbf{c}})$ matrice $N \times N$ che ha come elemento (nn') $\mathbf{1}\{\hat{c}_n = \hat{c}_{n'}\}$ e $\hat{\pi}_{nn'}$ stima Monte Carlo della *posterior similarity matrix*, una matrice $N \times N$ che ha come elemento (nn') la probabilità a posteriori che l'osservazione n e n' siano clusterizzate insieme. In Dahl and Newton (2007) si dimostra che minimizzare il criterio sopra è equivalente a minimizzare la stima Monte Carlo della perdita attesa Binder a posteriori con $a = b$. Questo significa che la partizione $\mathbf{c}^*$ ottimale per l'Equazione 9 è quella che minimizza

$$\sum_{n < n'} (\mathbf{1}(c_n = c_{n'}) - \pi_{nn'})^2.$$

In David B. Dahl and Müller (2022) viene approfondita la questione facendo notare come, nonostante la funzione di perdita *Binder* fosse stata introdotta con l'idea di pesare in modo diverso le due tipologie di errori, ogni implementazione ed applicazione fossero basate sul concetto di $a = b$. Pertanto viene fornita una versione invariante rispetto ad N della Binder loss ripristinando la flessibilità della potenziale scelta di valori differenti per a e b . Gli autori, in aggiunta, sostengono che questa scelta non è computazionalmente più complessa da implementare e porterebbe alla risoluzione del problema spesso

criticato della Binder, ossia la sovrastima del numero di cluster. Questa critica sembrerebbe essere una conseguenza dovuta dal fatto che nel passato si siano considerati solo costi uguali di miss-classificazione. Intuitivamente ci si aspetta che se si ponesse $a > b$ ciò porterebbe a una sovrastima del numero dei cluster, perché si metterebbe un costo più elevato a dividere un cluster incorrettamente rispetto a raggruppare in un solo cluster in modo errato due unità. Come dimostrato in David B. Dahl and Müller (2022) ciò non è vero, infatti, scegliendo un $a > b$ si è in grado di controllare la sovrastima del numero di cluster ed ottenere una partizione ottima stimata più parsimoniosa. Considerando il comportamento limite, per un b fissato, è evidente che per $a \rightarrow 0$ ogni osservazione comporrà un cluster a sé stante, mentre per $a \rightarrow \infty$ si avrà un cluster composto da tutte le osservazioni. La definizione della funzione di perdita *Binder* invariante rispetto ad N e con pesi a, b è dunque:

$$B(\mathbf{c}, \hat{\mathbf{c}}) = a \cdot \sum_{i=1}^{k_N} \left(\frac{n_{i+}}{N} \right)^2 + b \cdot \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{+j}}{N} \right)^2 - 2(a+b) \cdot \sum_{i=1}^{k_N} \sum_{j=1}^{\hat{k}_N} \left(\frac{n_{ij}}{N} \right)^2, \quad (10)$$

Si fa riferimento all'Equazione 10 come funzione di perdita di Binder generalizzata.

2.2 Variation of Information

La *Variation of Information* (*VI*) è stata introdotta in Meilă (2007) come misura di distanza tra due partizioni. Wade and Ghahramani (2018) sono stati i primi a considerare la *VI* come funzione di perdita per stimare una partizione casuale. La *VI* è costruita partendo dalla teoria dell'informazione e compara l'informazione in due raggruppamenti con l'informazione condivisa tra di essi. Formalmente è definita come

$$\begin{aligned} VI(\mathbf{c}, \hat{\mathbf{c}}) &= H(\mathbf{c}) + H(\hat{\mathbf{c}}) - 2I(\mathbf{c}, \hat{\mathbf{c}}) \\ &= - \sum_{i=1}^{k_N} \frac{n_{i+}}{N} \log \left(\frac{n_{i+}}{N} \right) - \sum_{j=1}^{\hat{k}_N} \frac{n_{+j}}{N} \log \left(\frac{n_{+j}}{N} \right) - 2 \sum_{i=1}^{k_N} \sum_{j=1}^{\hat{k}_N} \frac{n_{ij}}{N} \log \left(\frac{n_{ij}N}{n_{i+}n_{+j}} \right), \end{aligned}$$

con $\log$ ad indicare il logaritmo in base 2. I primi due termini misurano l'entropia calcolata sulle due partizioni (vera e stimata), che esprime la complessità nel prevedere in quale cluster appartiene un'unità statistica selezionata casualmente, conoscendone il partizionamento. Il terzo termine, invece, rappresenta la quantità di informazione condivisa tra le due partizioni che riduce l'incertezza nell'allocare una certa unità in un certo cluster di una partizione $\mathbf{c}$ nel momento in cui è nota la sua assegnazione sotto $\hat{\mathbf{c}}$. La *VI* ha un range tra 0 e $\log(N)$ e come la funzione di perdita Binder è invariante rispetto ad N .

Poiché $H(\mathbf{c})$ è costante quando si minimizza per $\hat{\mathbf{c}}$, Wade and Ghahramani (2018) fa notare che la partizione ottima, cioè quella che minimizza la perdita *VI* attesa a posteriori, è la stessa che determina il minimo della seguente espressione:

$$\begin{aligned} \hat{\mathbf{c}}^* &= \underset{\hat{\mathbf{c}}}{\operatorname{argmin}} \mathbb{E}[VI(\mathbf{c}, \hat{\mathbf{c}})|\mathcal{D}] \\ &= \underset{\hat{\mathbf{c}}}{\operatorname{argmin}} \sum_{n=1}^N \log \left(\sum_{n'=1}^N \mathbf{1}(\hat{c}_n = \hat{c}'_{n'}) \right) - 2 \sum_{n=1}^N \mathbb{E} \left[\log \left(\sum_{n'=1}^N \mathbf{1}(c_{n'} = c_n, \hat{c}_{n'} = \hat{c}_n) \right) | \mathcal{D} \right]. \quad (11) \end{aligned}$$

Il valore atteso in Equazione 11 può essere approssimato usando i campioni a posteriori $\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(M)}$ ottenuti tramite simulazioni MCMC. Valutare questo termine ha però una complessità computazionale pari a $O(MN^2)$ (con M iterazioni del MCMC), cioè il tempo necessario a calcolare quel termine cresce proporzionalmente al numero di campioni MCMC M e quadraticamente rispetto alla dimensione campionaria N ; ciò può rappresentare un serio vincolo computazionale. Per questo Wade and Ghahramani

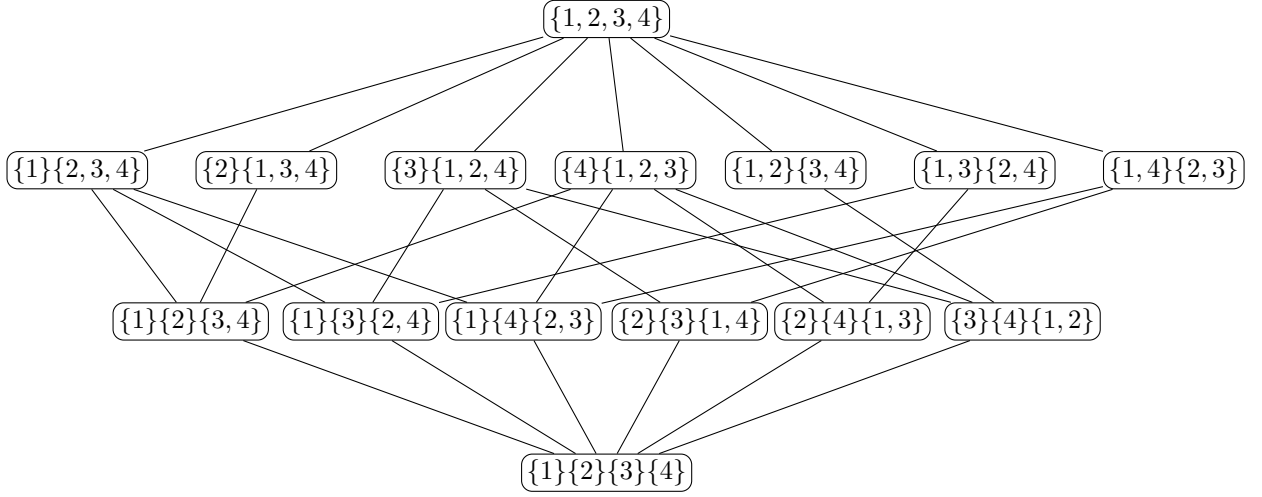


Figura 1: Diagramma di Hasse per un reticolo di partizioni per un campione di $N = 4$. La linea indica una relazione di copertura

(2018) propongono di applicare la disuguaglianza di Jensen per scambiare logaritmo e valore atteso calcolando un limite inferiore per l'Equazione 11 sulla perdita attesa a posteriori che è computazionalmente meno onerosa:

$$\hat{\mathbf{c}}^* = \underset{\hat{\mathbf{c}}}{\operatorname{argmin}} \sum_{n=1}^N \log \left(\sum_{n'=1}^N \mathbf{1}(\hat{c}_n = \hat{c}_{n'}) \right) - 2 \sum_{n=1}^N \log \left(\sum_{n'=1}^N \hat{\pi}_{nn'} \mathbf{1}(\hat{c}_{n'} = \hat{c}_n) \right)$$

con $\hat{\pi}_{nn'} = \frac{1}{M} \sum_{m=1}^M \mathbf{1}(c_n^{(m)} = c_{n'}^{(m)})$ elemento (nn') della *posterior similarity matrix* che può essere calcolata prima dell'ottimizzazione, rendendo così la complessità computazionale $O(N^2)$ per una data $\hat{\mathbf{c}}$. In David B. Dahl and Müller (2022) viene anche proposta una generalizzazione della funzione di perdita VI con la possibilità di specificare i pesi a e b che hanno un'interpretazione analoga a quella per la Binder loss, ma in questo progetto non è stata approfondita; di certo potrebbe essere uno spunto da cui iniziare per prossimi studi.

3 Algoritmi

In questa sezione sono spiegati i due algoritmi principali utilizzati per ottenere la partizione ottima dei dati. Per comprendere efficacemente il loro funzionamento vanno afferrati i concetti base di Teoria del reticolo, o più comunemente *Lattice Theory*, una sezione specifica della teoria degli ordini.

3.0.1 Teoria dell'ordine

La teoria del reticolo si basa su una rappresentazione grafica molto utile per rappresentare particolari ordinamenti tra partizioni: il diagramma di Hasse (rappresentato un esempio in Figura 1).

Per comprendere al meglio i due algoritmi e le *credible balls* è necessario comprendere alcune proprietà riguardanti lo spazio delle partizioni. In particolare, se considero lo spazio $\mathbf{C}$ e la relazione binaria $\leq$ su $\mathbf{C}$ viene definito il cosiddetto contenimento di insiemi. La partizione $\mathbf{C}$ avente la relazione $\leq$ è detto insieme parzialmente ordinato.

Per ogni coppia di partizioni $\mathbf{c}, \hat{\mathbf{c}} \in \mathbf{C}$, $\mathbf{c}$ è detto coperto da $\hat{\mathbf{c}}$, scritto come $\mathbf{c} \prec \hat{\mathbf{c}}$, se $\mathbf{c} < \hat{\mathbf{c}}$ e non c'è nessun $\hat{\hat{\mathbf{c}}} \in \mathbf{C}$ tale che $\mathbf{c} < \hat{\hat{\mathbf{c}}} < \hat{\mathbf{c}}$. La relazione di copertura è ben definita nel diagramma di Hasse in Figura 1. Prendendo per esempio la coppia di partizioni $\mathbf{c} = \{1, 2, 3\}, \{4\}$ e $\hat{\mathbf{c}} = \{1, 2\}, \{3\}, \{4\}$

è possibile affermare che $\hat{\mathbf{c}} \prec \mathbf{c}$. Il Diagramma di Hasse è infatti definito attraverso questa relazione d'ordine dove i nodi non sono altro che relazioni di copertura tra $\mathbf{c}$ e $\hat{\mathbf{c}}$. Lo spazio delle partizioni forma un reticolo, esemplificato da Figura 1. Da ciò consegue che ogni coppia di partizioni possiede un 'miglior' limite inferiore e un 'peggior' limite superiore. Questi limiti sono definiti in questo modo: per un sottoinsieme $\mathbf{S} \subseteq \mathbf{C}$, un elemento $\mathbf{c} \in \mathbf{C}$ è un *upper bound* per $\mathbf{S}$ se $\mathbf{s} \leq \mathbf{c}$ per ogni $\mathbf{s} \in \mathbf{S}$, e $\mathbf{c} \in \mathbf{C}$ è il miglior limite superiore per $\mathbf{S}$ se $\mathbf{c}$ è un limite superiore per $\mathbf{S}$ e $\mathbf{c} \leq \mathbf{c}'$ per ogni limite superiore $\mathbf{c}'$ di $\mathbf{S}$. I limiti inferiori e i migliori limiti inferiori sono similmente definiti. Infine si definisce l'operatore $\wedge$, chiamato *meet*, e $\vee$ chiamato *join*, tali che: $\mathbf{c} \wedge \hat{\mathbf{c}} = \text{g.l.b.}(\mathbf{c}, \hat{\mathbf{c}})$ e $\mathbf{c} \vee \hat{\mathbf{c}} = \text{l.u.b.}(\mathbf{c}, \hat{\mathbf{c}})$. La teoria degli ordini è molto importante poiché entrambe la metriche *VI* e *Binder* descritte rispettivamente in Sezione 2.1 e Sezione 2.2 sono allineate con il reticolo delle partizioni. In particolare, entrambe le metriche sono verticalmente allineate nel diagramma di Hasse se: $\hat{\mathbf{c}}$ è connesso fino a $\hat{\mathbf{c}}$ e $\hat{\mathbf{c}}$ è connesso fino a $\mathbf{c}$, allora la distanza tra $\hat{\mathbf{c}}$ e $\mathbf{c}$ è la somma verticale tra le distanza tra $\hat{\mathbf{c}}$ e $\hat{\mathbf{c}}$ e $\hat{\mathbf{c}}$ e $\mathbf{c}$. Inoltre, le due metriche sono orizzontalmente allineate, ossia la distanza tra due partizioni è la somma orizzontale della distanza tra queste e il meet delle due partizioni. In particolare queste due proprietà delle metriche sono espresse in Equazione 12 e 13

$$d(\mathbf{c}, \hat{\mathbf{c}}) = d(\mathbf{c}, \hat{\mathbf{c}} \wedge \mathbf{c}) + d(\hat{\mathbf{c}}, \hat{\mathbf{c}} \wedge \mathbf{c}) \quad (12)$$

$$d(\mathbf{c}, \hat{\mathbf{c}}) = d(\mathbf{c}, \hat{\mathbf{c}} \vee \mathbf{c}) + d(\hat{\mathbf{c}}, \hat{\mathbf{c}} \vee \mathbf{c}) \quad (13)$$

Queste proprietà implicano inoltre che la distanza tra una coppia di partizioni qualsiasi è sempre:

$$d(\mathbf{c}, \hat{\mathbf{c}}) \leq d(\mathbf{1}, \mathbf{0}) \quad (14)$$

dove $\mathbf{1}$ è la partizione dove tutte le osservazioni sono contenute nello stesso cluster, mentre $\mathbf{0}$ indica la partizione dove tutte le osservazioni si trovano in cluster differenti. Da ciò consegue che:

$$VI(\mathbf{c}, \hat{\mathbf{c}}) \leq \log(N), B(\mathbf{c}, \hat{\mathbf{c}}) \leq 1 - \frac{1}{N}. \quad (15)$$

In entrambi i casi il limite superiore della distanza tra le partizioni dipende dalla numerosità campionaria N . Tuttavia il loro approccio per $N \mapsto +\infty$ è differente: *VI* tende a $+\infty$, mentre *Binder* a 1. Un'ultima proprietà riguardante le metriche *VI* e *B* è che, le partizioni più vicine ad una partizione $\mathbf{c}$ sono:

- se $\mathbf{c}$ contiene almeno due cluster di ampiezza 1 e almeno un cluster di ampiezza 2, le partizioni che uniscono qualsiasi coppia di cluster di dimensione uno e le partizioni che dividono qualsiasi cluster di dimensione due;
- se $\mathbf{c}$ contiene almeno due cluster di dimensione uno e nessun cluster di dimensione due, le partizioni che uniscono qualsiasi coppia di cluster di dimensione uno;
- se $\mathbf{c}$ contiene al massimo un cluster di dimensione uno, le partizioni che dividono il cluster più piccolo di dimensione maggiore di uno in un singolo e un cluster con le osservazioni rimanenti del cluster originale.

Ora che sono state introdotte le fondamenta di teoria dell'ordinamento è possibile osservare gli algoritmi proposti da Wade and Ghahramani (2018) e da Dahl et al. (2022) nelle prossime sezioni.

3.1 Algoritmo Greedy Search

In Wade and Ghahramani (2018) è proposto un algoritmo di ricerca Greedy ('avido') per ottenere la partizione ottima $\mathbf{c}^*$ basandosi sul diagramma di Hasse. L'algoritmo può essere utilizzato sia con *VI* che con *Binder*. In particolare, data una qualche partizione $\hat{\mathbf{c}}$, considero la l -esima partizione più vicina che copre $\hat{\mathbf{c}}$ e la l -esima partizione più vicina che $\hat{\mathbf{c}}$ copre. Di default l è scelto pari a due volte il numero di righe della posterior similarity matrix, ossia due volte la numerosità campionaria. Qui, la distanza utilizzata per determinare la partizione più vicina, dipende dalla funzione di perdita designata.

Successivamente la perdita attesa a posteriori $\mathbb{E}[L(\mathbf{c}, \hat{\mathbf{c}})|\mathcal{D}]$ viene calcolata per tutte le partizioni proposte $\hat{\mathbf{c}}$ e si cerca di trovare un punto di ottimo in direzione del minimo atteso. In questo modo si trova la partizione $\mathbf{c}^*$ che minimizza la perdita attesa a posteriori. L'algoritmo finisce quando non si hanno significative riduzioni della perdita attesa oppure al raggiungimento del numero massimo di iterazioni prestabilito. Per ogni iterazione la complessità computazionale è pari a $O(lN^2)$, dove N è la numerosità campionaria.

In sintesi l'algoritmo greedy è una procedura che effettua piccoli aggiornamenti degli ottimi locali ad ogni step al fine di arrivare alla soluzione ottimale, ossia la minimizzazione ideale della funzione di perdita a posteriori.

Un vantaggio dell'algoritmo di ricerca greedy rispetto al semplice restringersi alle partizioni esplorate nella catena MCMC è che possono essere considerate anche strutture di clustering non esplorate nelle simulazioni MCMC. Infatti, in quasi tutti gli esempi simulati e reali, la stima del clustering non si trova tra le partizioni campionate e porta a una perdita attesa inferiore rispetto a qualsiasi partizione campionata.

Una pecca, come mostrata in Dahl et al. (2022), è la sua dipendenza dalla partizione iniziale, specialmente dato che l'algoritmo può rimanere incastrato in problemi di minimi locali. Un'estensione dell'algoritmo di Wade and Ghahramani (2018) è stato proposto da Rastelli and Friel (2018) che cerca di risolvere il problema dei minimi locali inizializzando l'algoritmo in modo randomico. In particolare questa partizione random deve essere composta da tanti piccoli cluster e iterativamente riassegnare ciascuno di essi ad cluster esistente, dove la riassegnazione viene eseguita cercando di minimizzare la stima Monte Carlo della perdita attesa. Rastelli and Friel (2018) afferma, infatti, che un numero di ripetizione randomiche maggiori può aumentare le possibilità di trovare una risposta migliore e ottimizzare i problemi di minimalità locale. L'algoritmo Greedy Search di Rastelli and Friel (2018) viene anche chiamato Stochastic Greedy Search.

3.1.1 Crediball Balls

In Wade and Ghahramani (2018) è evidenziato come, nella letteratura, la quantificazione dell'incertezza per la struttura di clustering fosse descritta principalmente attraverso una heat map della stima della matrice di similarità a posteriori, nonostante non vi fosse una precisa misura di quanta incertezza fosse rappresentata dalla matrice stessa. Per questo motivo viene proposta la costruzione di una credible ball di livello di confidenza $1 - \alpha$, con $\alpha \in [0, 1]$ piccolo, definita come

$$Ball_{\epsilon^*}(\mathbf{c}^*) = \{\mathbf{c} : d(\mathbf{c}^*, \mathbf{c}) \leq \epsilon^*\}$$

con ϵ^* il più piccolo $\epsilon \geq 0$ tale che $P(Ball_{\epsilon^*}(\mathbf{c}^*)|\mathcal{D}) \geq 1 - \alpha$. In sostanza, la credible ball è la più piccola palla che circonda $\mathbf{c}^*$ con probabilità a posteriori pari almeno a $1 - \alpha$ e che riflette l'incertezza nella stima puntuale. Si crede che con probabilità $1 - \alpha$ la partizione sia entro una distanza ϵ^* dalla stima puntuale $\mathbf{c}^*$ considerati i dati. La credible ball può essere definita basandosi su qualsiasi metrica sullo spazio delle partizioni come la perdita *VI* e *Binder*.

La stima per ϵ^* , e quindi per la credible ball, si ottiene dai risultati del campionamento MCMC. In primo luogo si calcola la distanza fra tutti i campioni MCMC e $\mathbf{c}^*$, successivamente per ogni $\epsilon > 0$,

$$P(Ball_{\epsilon^*}(\mathbf{c}^*)|\mathcal{D}) = \mathbb{E}[\mathbf{1}(VI(\mathbf{c}^*, \mathbf{c}) \leq \epsilon)|\mathcal{D}] \approx \frac{1}{M} \sum_{m=1}^M \mathbf{1}(d(\mathbf{c}^*, \mathbf{c}^{(m)}) \leq \epsilon),$$

e ϵ^* è il più piccolo $\epsilon > 0$ tale che $\frac{1}{M} \sum_{m=1}^M \mathbf{1}(d(\mathbf{c}^*, \mathbf{c}^{(m)}) \leq \epsilon) \geq 1 - \alpha$.

Per caratterizzare la credible ball si definiscono successivamente i limiti verticali e orizzontali, che descrivono gli estremi di quest'ultima. Il limite verticale superiore consiste nella partizione nella credible ball con il minor numero di cluster che è più distante da $\mathbf{c}^*$. Il limite verticale inferiore consiste nella partizione nella credible ball con il maggior numero di cluster più distante da $\mathbf{c}^*$. Il limite orizzontale consiste nella partizione della credible ball che è più distante da $\mathbf{c}^*$.

In conclusione, in Wade and Ghahramani (2018) si confrontano le 95% crediball ball con la *posterior similarity matrix*, concludendo che quest'ultima sotto rappresenta l'incertezza. Meritevoli di attenzione

sono anche alcuni commenti interessanti a Wade and Ghahramani (2018) come quello di Nipoti and Shen (2018), il quale si concentra su formulazioni alternative per le credible ball. Diversamente da come sono state definite da Wade and Ghahramani (2018) si potrebbe evitare di centrare la credible ball in una qualsiasi partizione data, suggerendo invece un approccio basato sull'entropia. In questo modo si ordinerebbe il campione MCMC delle partizioni attraverso una certa misura, per esempio l'entropia, definendole in questo modo:

$$Ball_{\epsilon^*}^{(H)} = \{\mathbf{c} : H_L \leq H(\mathbf{c}) \leq H_U\},$$

con $H_L := H(\mathbf{c}_H^{(l)})$ e $H_U := H(\mathbf{c}_H^{(u)})$ e con $\mathbf{c}_H^{(l)}$ e $\mathbf{c}_H^{(u)}$ quantili empirici dell'insieme ordinato delle partizioni osservate, tali che $\frac{1}{M} \sum_{m=1}^M \mathbf{1}\{H_L \leq H(\mathbf{c}^{(m)}) \leq H_U\} \geq 1 - \alpha$. Nello studio ci si è limitati all'implementazione della credible ball di Wade and Ghahramani (2018), ma in un futuro sviluppo potrebbe rappresentare di certo un aspetto da approfondire.

3.2 Algoritmo Salso

L'algoritmo Salso, come il Greedy Search, cerca la partizione ottima sfruttando le simulazioni MCMC e la perdita attesa. Come l'algoritmo introdotto da Rastelli and Friel (2018), il Salso è un algoritmo di ricerca stocastica, con particolari parametri di impostazione. L'algoritmo è composto da quattro fasi:

1. inizializzazione;
2. ottimizzazione sweetening;
3. aggiornamenti zealous;
4. registrazione finale.

L'algoritmo Salso propone due metodi differenti per l'**inizializzazione**. Il primo metodo viene chiamato allocazione sequenziale. Ogni osservazione viene allocata una alla volta a un cluster esistente o ad uno nuovo. L'ordine di assegnazione è dato da una permutazione estratta uniformemente da tutte le possibili permutazioni. L'assegnazione avviene minimizzando la perdita attesa a posteriori ottenuta attraverso le stime Monte Carlo, ignorando le osservazioni non ancora allocate. Il processo continua fino all'utilizzo di tutte le osservazioni e il numero di cluster può crescere illimitatamente, a meno che si imponga come parametro di inizializzazione un numero massimo di cluster k_N .

Nel secondo metodo di inizializzazione, le etichette dei cluster $1, \dots, k_N$ sono ottenute attraverso un campionamento uniforme. Il metodo di inizializzazione viene scelto casualmente. E' inoltre possibile preferire una inizializzazione, rispetto ad un'altra, specificando la probabilità di selezionare l'allocazione sequenziale, indicata come p_{SA} , il cui valore predefinito è 0,5.

Una volta inizializzata la partizione, il prossimo step è lo **sweetening**, ossia la fase di ottimizzazione. In questa fase, ogni osservazione viene riallocata, una alla volta, in maniera casuale. Il processo è molto simile all'allocazione sequenziale del metodo di inizializzazione, con la differenza che ora vengono considerate tutte le osservazioni e le loro allocazioni. Quindi, ogni osservazione, una alla volta e nell'ordine di una permutazione ottenuta tramite campionamento uniforme tra tutte le permutazioni, viene rimossa dal proprio cluster e riallocata in un cluster esistente o ad uno nuovo, in modo da minimizzare la stima Monte Carlo della perdita attesa. Questo processo viene ripetuto fino a quando non si verificano cambiamenti significativi dopo un passaggio su tutte le N osservazioni.

Nella terza fase, la fase degli aggiornamenti **zealous**, si cerca di superare i problemi di minimo locale. Si effettuano aggiornamenti computazionalmente più intensivi. E' possibile impostare un numero massimo di questi aggiornamenti, denotato con $n_{maxZealous}$. In modo casuale si distrugge un cluster, dopo aver registrato lo stato attuale, si rimuovono tutti i suoi componenti e li si riallocano secondo il metodo dell'allocazione sequenziale. Una volta che tutti gli elementi sono riassegnati, si confronta la stima Monte Carlo della perdita attesa a posteriori della nuova partizione con quella precedente. Se non viene trovata alcuna miglione, l'aggiornamento viene abbandonato e lo stato torna alla versione precedente.

Algorithm 1 Pseudocodice per l'Algoritmo SALSO. Sia n_{runs} il numero di esecuzioni, p_{SA} la probabilità di allocazione sequenziale, k_d il numero massimo di cluster e $n_{maxZealous}$ il numero massimo di aggiornamenti zealous. Nota: quando si considera "tutti i cluster esistenti e un nuovo cluster", l'algoritmo non considera un nuovo cluster se ciò portasse a uno stato in cui il numero di cluster supera il massimo k_d .

```

1: for  $i = 1, \dots, n_{runs}$  do
2:   // Fase di inizializzazione
3:   if  $\text{Uniform}(0,1) < p_{SA}$  then
4:     // Effettua l'inizializzazione sequenziale
5:     Esegui una permutazione casuale  $(\sigma_1, \dots, \sigma_n)$  di  $\{1, \dots, n\}$ .
6:     for  $\sigma$  in  $(\sigma_1, \dots, \sigma_n)$  do
7:       for  $c$  in tutti i cluster esistenti e un nuovo cluster do
8:         Calcola la perdita attesa a posteriori se  $\sigma$  venisse aggiunto al cluster  $c$ .
9:       end for
10:      Alloca  $\sigma$  al cluster che minimizza la perdita attesa a posteriori.
11:    end for
12:  else
13:    // Effettua l'inizializzazione casuale
14:    Campiona una partizione casuale con al massimo  $k_d$  cluster, ad es., sample(1:k_d, n, replace=TRUE) in R.
15:  end if
16:  // Fase di ottimizzazione
17:  while TRUE do
18:    Esegui una permutazione casuale  $(\sigma_1, \dots, \sigma_n)$  di  $\{1, \dots, n\}$ .
19:    for  $\sigma = (\sigma_1, \dots, \sigma_n)$  do
20:      Rimuovi  $\sigma$  dal suo cluster.
21:      for  $c$  in tutti i cluster esistenti e un nuovo cluster do
22:        Calcola la perdita attesa a posteriori se  $\sigma$  venisse aggiunto al cluster  $c$ .
23:      end for
24:      Alloca  $\sigma$  al cluster che minimizza la perdita attesa a posteriori.
25:    end for
26:    if la partizione è invariata then
27:      break
28:    end if
29:  end while
30:  // Fase di aggiornamenti zealous
31:  for  $c'$  in un ordine casuale di al massimo  $n_{maxZealous}$  cluster do
32:    Registra la partizione corrente come  $\hat{\rho}$ .
33:    Distruggi il cluster  $c'$  rimuovendo (deallocando) tutti i suoi elementi.
34:    for  $\sigma$  in un ordine casuale degli elementi deallocati do
35:      for  $c$  in tutti i cluster esistenti e un nuovo cluster do
36:        Calcola la perdita attesa a posteriori se  $\sigma$  venisse aggiunto al cluster  $c$ .
37:      end for
38:      Alloca  $\sigma$  al cluster che minimizza la perdita attesa a posteriori.
39:    end for
40:    if  $\hat{\rho}$  ha una perdita attesa a posteriori minore rispetto alla partizione corrente then
41:      Ritorna alla partizione  $\hat{\rho}$ .
42:    end if
43:  end for
44:  // Fase di registrazione
45:  Registra la partizione corrente come  $\hat{\rho}_i$  e annota la sua perdita attesa a posteriori.
46: end for
47: Restituisci la  $\hat{\rho}_i$  con la minima perdita attesa a posteriori.

```

Per concludere, nella quarta fase del Salso, si registra la stima Monte Carlo della perdita attesa a posteriori per l'iterazione dell'algoritmo corrente. C'è da notare che questo algoritmo è facilmente 'parallelizzabile' dato che ogni iterazione è indipendente dalle altre. Tra tutte le n_{runs} partizioni candidate, viene riportata come stima della partizione quella con la stima Monte Carlo più bassa della perdita attesa a posteriori. Un'osservazione utile da menzionare è che l'algoritmo proposto da Dahl et al. (2022) non è altro che una generalizzazione dell'algoritmo Greedy di Rastelli and Friel (2018). Per ottenere quest'ultimo basta infatti porre $p_{SA} = 0$, $n_{maxZealous} = 0$, $n_{runs} = 1$. In Algoritmo 1 è rappresentato uno schema dell'algoritmo Salso.

3.3 Comparazione computazionale degli algoritmi

In questa sezione vengono comparati gli algoritmi in termini computazionali. Durante lo studio di simulazione in Sezione 4.1, verranno inoltre confrontati i tempi computazionali per eseguire l'algoritmo sull'ordinatore. Come già accennato in Sezione 3.1, la complessità dell'algoritmo *Greedy Search* di Wade and Ghahramani (2018) è pari a $O(lN^2)$. L'algoritmo *Salso* invece ha una complessità pari a $O(Hk_dk_hN)$, dove H è il numero di simulazioni MCMC, k_d il numero massimo di cluster, inserito dall'utente, k_h il numero massimo di cluster osservato durante il campionamento MCMC e N il numero di osservazione presenti nei dati. La complessità dell'algoritmo *Greedy Search* proposto invece da Rastelli and Friel (2018) è invece pari a $O(HN^3)$.

4 Esempi

In questa sezione abbiamo sviluppato uno studio su dati simulati per comparare le due funzione di perdita: *Binder* e *VI* con gli algoritmi proposti da Wade and Ghahramani (2018) e Dahl et al. (2022), rappresentando inoltre l'incertezza ottenuta nelle stime delle partizioni attraverso le credible balls. Successivamente abbiamo implementato l'algoritmo che durante la simulazione è risultato essere migliore in termini di efficacia e efficienza su due set di dati reali riguardanti alcune statistiche nazionali.

4.1 Studio di simulazione

Per lo studio di simulazione abbiamo deciso di generare $n = 300$ osservazioni da una mistura di normali trivariate composta da 5 componenti:

$$X_i \stackrel{iid}{\sim} \sum_{j=1}^5 \frac{1}{5} \mathcal{N} \left(\begin{bmatrix} \mu_{j1} \\ \mu_{j2} \\ \mu_{j3} \end{bmatrix}, \begin{bmatrix} \sigma_j^2 & 0 & 0 \\ 0 & \sigma_j^2 & 0 \\ 0 & 0 & \sigma_j^2 \end{bmatrix} \right). \quad (16)$$

In particolare in Tabella 1 sono rappresentate le medie, mentre in Tabella 2 le varianze di ciascuna componente.

	j = 1	j = 2	j = 3	j = 4	j = 5
μ_1	-2	2	-2	2	0
μ_2	-2	-2	2	2	0
μ_3	-2	2	-2	2	0

Tabella 1: Valori del vettore delle medie per ciascuna componente

In Figura 2 sono rappresentati i punti simulati distinti per ciascuna componente del modello.

Per modellare i dati, al fine di effettuare clustering e implementare i due algoritmi, abbiamo utilizzato un modello mistura del processo di Dirichlet con kernel gaussiano. In particolare abbiamo utilizzato, prendendo la notazione espressa in Corradin et al. (2021), un modello *Diagonal Location Scale*, dove la matrice di varianza e covarianza del kernel è diagonale. Il modello può quindi essere scritto come:

	j = 1	j = 2	j = 3	j = 4	j = 5
σ_j^2	1.00	1.25	1.00	1.25	0.50

Tabella 2: Valori delle varianze σ_j^2 per ciascuna componente

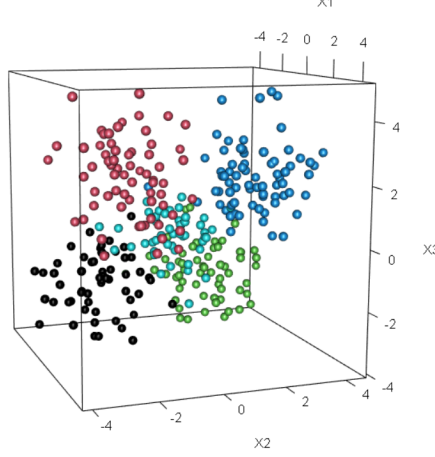


Figura 2: Scatterplot tridimensionale dei dati simulati per $N = 300$

$$X_i | \tilde{P} \stackrel{\text{iid}}{\sim} \int \mathcal{N} \left(\begin{bmatrix} \mu_1 \\ \mu_2 \\ \mu_3 \end{bmatrix}, \begin{bmatrix} \sigma_1^2 & 0 & 0 \\ 0 & \sigma_2^2 & 0 \\ 0 & 0 & \sigma_3^2 \end{bmatrix} \right) d\tilde{P}(\tilde{\mu}, \Sigma). \quad (17)$$

con $\tilde{P} \sim DP(\alpha, G_0)$.

Dato le assunzioni fatte in precedenza è possibile riscrivere 17 nel seguente modo:

$$\tilde{f}(x) = \sum_{j=1}^{\infty} \pi_j \prod_{r=1}^3 \phi(x_r; \tilde{\mu}_{jr}, \tilde{\sigma}_{jr}^2), \quad (18)$$

dove x_r è la r -esima componente di $\mathbf{X}$ e $\tilde{\Sigma}_j$ è diagonale. Per questo modello è stata considerata la misura base G_0 coniugata. In particolare G_0 non è altro che un triplo prodotto di normali-gamme inverse indipendenti con iperparametri $(m0_r, k0_r, a0_r, b0_r)$. La misura base ha quindi densità:

$$g_0(\mu_1, \mu_2, \mu_3, \sigma_1^2, \sigma_2^2, \sigma_3^2) \propto \prod_{i=1}^3 \sqrt{\frac{k0_i}{\sigma_i^2}} \exp\left(-\frac{c_i}{2\sigma_i^2}(\mu_i - m0_i)^2\right) (\sigma_i^2)^{-a0_i-1} \exp\left(-\frac{b0_i}{\sigma_i^2}\right). \quad (19)$$

Gli iperparametri sono stati elicitati nel seguente modo: $m0$ vettore tridimensionale di zeri, $k0$ vettore tridimensionale di $1/2$, $a0$ vettore tridimensionale di 2 e $b0$ vettore di 1. Il parametro di concentrazione α viene stabilito da una hyperprior Gamma(1,1) come consigliato in Escobar and West (1995).

L'algoritmo utilizzato per simulare la distribuzione a posteriori è il Marginal Gibbs Sampling (Neal (2000)) con 10000 iterazioni ed un periodo di burn in pari a 1000. In Figura 3 è rappresentato il grafico relativo alla convergenza osservata sull'entropia. In particolare, la convergenza delle catene, è stata monitorata sul numero di cluster ottenuti attraverso il campionamento MCMC e sull'entropia, la cui formula è:

$$H(\mathbf{c}) = \log(N) - \frac{1}{N} \sum_{j=1}^{k_N} n_j \log(n_j). \quad (20)$$

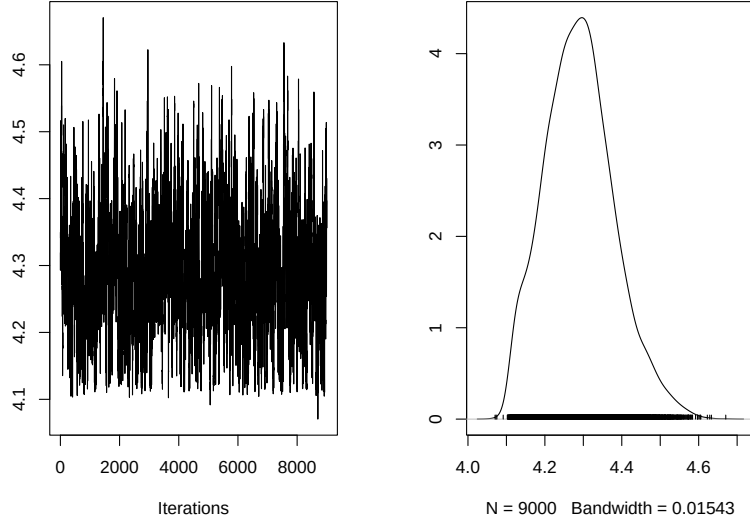


Figura 3: Grafici di convergenza MCMC relativi all'entropia assegnata a ciascuna partizione visitata

In Figura 4 è invece rappresentata la distribuzione a posteriori del numero di cluster ottenuta attraverso il campionamento MCMC.

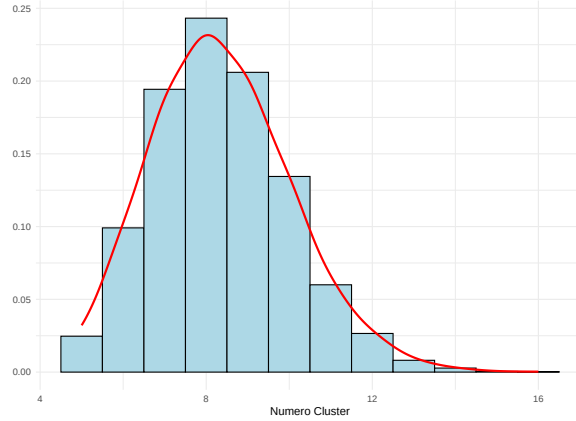


Figura 4: Stima della distribuzione a posteriori del numero di cluster

La convergenza è stata inoltre monitorata attraverso l'utilizzo del Geweke test (Geweke (2004)), che ha esposto risultati positivi per entrambi i metodi sperimentati.

Inizialmente è stato implementato l'algoritmo Greedy Search utilizzando come funzione di perdita la *Variation of Information*. Il risultato prodotto è una partizione composta da 5 cluster con le numerosità espresse in Tabella 3.

In Figura 6 sono rappresentate le proiezioni bidimensionali dei cluster ottenuti con la *Variation of Information*. I colori rappresentano i cinque diversi gruppi e sono stati scelti in modo da avere una corrispondenza con la Figura 2 che rappresenta le vere componenti del modello simulato. Le palle di credibilità al 95% stimate dal Greedy Search mostrano come limite orizzontale un totale di 9 cluster, ossia la partizione con la maggiore distanza dalla partizione ottima, come limite verticale superiore 5 cluster, ossia la partizione nella palla con il minor numero di cluster più lontani dalla partizione ottima.

	j = 1	j = 2	j = 3	j = 4	j = 5
Veri cluster	63	65	54	74	44
VI Greedy Search cluster	56	66	58	72	48

Tabella 3: Numerosità n_j per ciascun cluster

e come limite inferiore verticale 16 cluster, ossia la partizione nella palla con il maggior numero di cluster lontani tra loro. In Figura 5 sono mostrate, in maniera grafica, le stime ottenute attraverso le credible balls. Graficamente sono state rappresentate soltanto le prime due componenti del modello tridimensionale per questioni di rappresentazione. Come ben visibile le palle di credibilità al 95% comunicano che la partizione ottima è composta, secondo la funzione di perdita VI, nel 95% dei casi al massimo da 16 cluster e al minimo da 5. Se si osserva, in Figura 5, il limite verticale superiore, si può notare come alcuni punti, che si trovano in zone di confine tra due o più cluster, vengano raggruppati in maniera differente rispetto al partizione ottima trovata dall'algoritmo. Ne sono un esempio alcuni punti appartenenti al cluster blu che risiedono al confine con il cluster verde. Nella partizione ottima questi punti appartengono al cluster verde, mentre nel limite verticale superiore della palla di credibilità al blu.

Osservando invece, il limite orizzontale e il limite verticale inferiore, si può osservare come la maggior parte dei cluster sia concentrato nel centro del piano cartesiano. Questo può essere spiegato dal fatto che quei punti, trovandosi in una terra di incertezza fra più cluster, durante l'algoritmo tendono a formare nuovi gruppi composti da poche osservazioni. Questo, grazie al lavoro svolto dalla funzione di perdita, porta a creare delle partizioni che sono relativamente lontane da quella ottima. Grazie alle credible balls siamo stati in grado di dare una misura dell'incertezza fornita dai cluster ottenuti attraverso l'algoritmo ed ad osservare quali sono i punti più difficilmente raggruppabili.

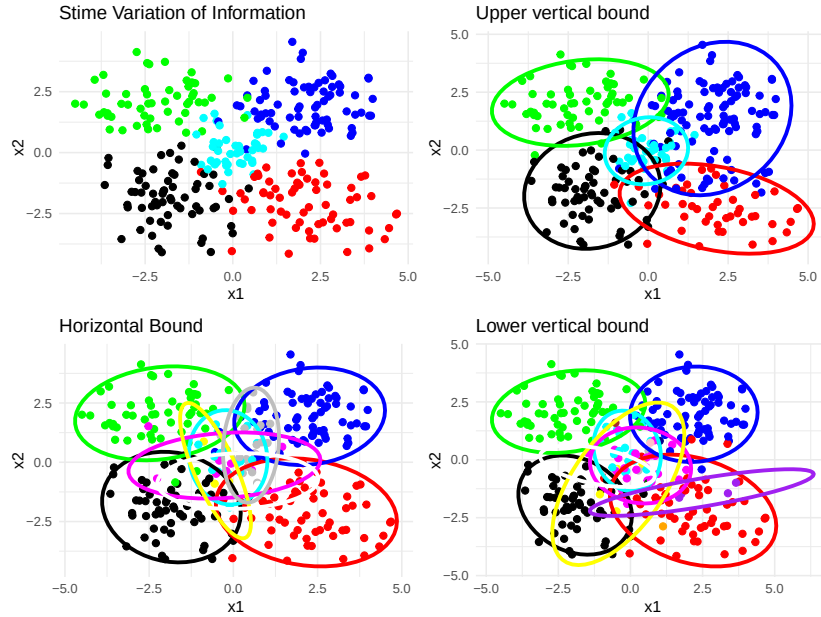


Figura 5: Nella prima Figura in alto a destra è rappresentata la partizione ottima ottenuta attraverso l'algoritmo Greedy e la funzione di perdita VI; nella seconda Figura in alto a destra è rappresentato il limite verticale superiore ottenuto con le credible balls e formato da 5 cluster, in basso a sinistra il limite orizzontale composto da 9 cluster e in basso a destra il limite verticale inferiore composto da 16 cluster

Successivamente è stato eseguito l'algoritmo Greedy Search con funzione di perdita *Binder* ottenendo come partizione ottima 24 cluster. Come ben visibile, il numero di cluster è largamente superiore. E' evidente come la funzione di perdita *Binder* tenda a creare nuovi cluster composti da poche osservazioni che si trovano ai bordi dei veri. Quando un punto è in una posizione incerta, la *Binder* loss crea un nuovo cluster, andando a sovrastimare quello che è il numero reale. Questo verrà evidenziato successivamente, quando verranno confrontati tutti gli algoritmi implementati per diverse numerosità di simulazione.

Nella Tabella riassuntiva 4 è rappresentata una sintesi dei risultati ottenuti dalle credible balls attraverso l'algoritmo greedy per le due funzioni di perdita: *Binder* e *Loss*.

Loss	Upper		Lower		Horizontal	
	k_N^u	$d(c^*, c_u)$	k_N^l	$d(c^*, c_l)$	k_N^h	$d(c^*, c_h)$
Binder	5	1.23	16	1.11	12	1.28
VI	5	1.03	16	1.07	9	1.21

Tabella 4: Tabella riassuntiva dei risultati ottenuti attraverso le credible balls

Come riportato in precedenza, la *Binder* loss trova come partizione ottima 24 cluster. Il fatto che il numero di cluster della partizione ottima sia maggiore rispetto al numero di cluster nei limiti delle credible ball può essere interpretato come una riflessione dell'incertezza nelle stime dei cluster in relazione alla probabilità e alla varianza associata alla partizione ottima. Poiché le credible balls rappresentano le regioni in cui le stime sono considerate credibili con il 95% di probabilità, queste aree possono "smussare" o aggregare i cluster, riducendo il numero di cluster separati nelle partizioni incerte. Le credible balls tendono a raggruppare i dati che sono più incerti o situati ai margini dei cluster, riducendo, in questo caso, la granularità della partizione. E' possibile notare come i limiti ottenuti dalle credible balls sono molto simili per le due funzioni di perdita, ciò che non accade per la partizione ottima dove la funzione *VI* sembra essere più precisa nella creazione dei cluster considerato l'ambiente supervisionato dove ci troviamo.

Infine è stato implementato anche l'algoritmo Salso, rispettivamente per le funzioni di perdita: *Binder* simmetrica, generalizzata e *VI*. I pesi da attribuire alle due differenti componenti della *Binder* generalizzata sono stati scelti in modo da ottenere 5 cluster, così da avere conformità con la distribuzione dei dati simulata. In questo caso il valore di a ottimo è risultato essere pari a 1.466667 (riferito a Equazione 10). L'algoritmo Salso, applicato alla funzione di perdita *VI*, ha prodotto come risultato un numero di cluster pari a 5; con *Binder* simmetrica pari a 16 e con *Binder* generalizzata e il parametro a scelto in maniera ottima, pari a 5. L'algoritmo Salso è stato impostato con parametri pari a: $nRuns = 16$, $maxZealous = 10$, $p_{SA} = 0.5$, non impostando il numero massimo di cluster da osservare.

Come ultimo passo dello studio di simulazione si è deciso di osservare come gli algoritmi avrebbero risposto all'aumentare della numerosità del campione simulato e come le funzioni di perdita avrebbe reagito di conseguenza. N , ossia la numerosità campionaria dei dati simulati, è stata testata sui valori 300 (esempio precedente), 600, 900 e 1600. In Tabella 5 sono rappresentati i risultati ottenuti, differenziati per la numerosità campionaria prescelta. Oltre ai risultati proposti in termini di numero dei cluster formati (k_N), numero di osservazione 'mal clusterizzate' (N_i), *Binder* loss valutata tra la partizione stimata e quella vera ($B(\hat{c}, c)$), *VI* valutata tra la partizione stimata e quella vera ($VI(\hat{c}, c)$), Adjusted Rand Index (*ARI*), sono stati valutati anche i tempi computazionali impiegati dall'algoritmo. L'Adjusted Rand Index non è altro che una metrica per valutare la qualità di un clustering, confrontando i risultati di un algoritmo con una partizione di riferimento. L'Adjusted Rand Index è una misura corretta del Rand Index, strettamente collegato con la metrica *Binder*. L'ARI introduce una normalizzazione in modo che il valore si trovi tra -1 ed 1. Valori di ARI vicino al 1 indicano che la partizione ottenuta è molto simile alla partizione reale.

Come ben visibile, all'aumentare della numerosità di simulazione l'algoritmo Greedy con funzione di perdita *Binder* tende a peggiorare, andando a stimare dai 24 cluster nel caso di $N = 300$, ai 41 nel caso di $N = 1800$. Questo porta a pensare che al crescere di N più punti siano allocati sui bordi dei cluster provocando maggiore incertezza e la *Binder* loss, sovrastimando il numero dei cluster, è

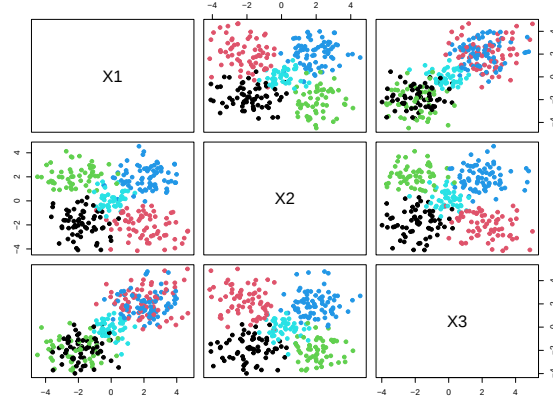


Figura 6: Proiezioni bidimensionali dei cluster ottenuti attraverso l'algoritmo Greedy Search e la funzione di perdita VI

N	Algoritmo	k_N	N_i	$B(\hat{c}, c)$	$VI(\hat{c}, c)$	ARI	Sys TIme
$N = 300$	Greedy VI	5	13	0.0326	0.5323	0.8985	25.0549
	Greedy Binder	24	27	0.0384	0.7658	0.8753	32.1067
	Salso VI	5	13	0.0326	0.5323	0.8985	4.2243
	Salso Binder (a = 1)	16	25	0.0377	0.7310	0.8787	132.0422
	Salso Binder (a = 1.46)	5	14	0.0352	0.5541	0.8909	20.0045
$N = 600$	Greedy VI	5	31	0.0391	0.6605	0.8777	117.7035
	Greedy Binder	24	279	0.0413	0.8061	0.8666	190.9277
	Salso VI	5	34	0.0428	0.7048	0.8660	16.1122
	Salso Binder (a = 1)	16	45	0.0419	0.7849	0.8654	341.1415
	Salso Binder (a = 1.46)	5	34	0.0424	0.7217	0.8672	59.5962
$N = 900$	Greedy VI	5	48	0.0408	0.7207	0.8724	420.2393
	Greedy Binder	32	282	0.0422	0.8815	0.8639	455.5567
	Salso VI	5	52	0.0443	0.7638	0.8614	35.9570
	Salso Binder (a = 1)	17	67	0.0430	0.8666	0.8622	432.0280
	Salso Binder (a = 1.46)	5	51	0.0433	0.7641	0.8645	151.4777
$N = 1600$	Greedy VI	5	87	0.0420	0.7399	0.8688	3692.0178
	Greedy Binder	41	637	0.0408	0.8563	0.8690	2967.4418
	Salso VI	5	84	0.0408	0.7079	0.8725	66.1045
	Salso Binder (a = 1)	17	110	0.0414	0.8419	0.8678	1194.4606
	Salso Binder (a = 1.46)	5	86	0.0416	0.7328	0.8701	360.4050

Tabella 5: Tabella riassuntiva dei risultati prodotti dagli algoritmi messi a confronto al variare della numerosità campionaria N

sensibile all'aumento della numerosità campionaria. Anche il numero di osservazioni incorrettamente classificate aumenta all'aumentare di N . Tuttavia, se si tiene in considerazione il rapporto percentuale tra le osservazioni erroneamente clusterizzate ed N , si può osservare come per la funzione di perdita VI questo rapporto resti costante (circa 4%), mentre per *Binder* passa dal 9% con $N = 300$ al 35% per $N = 1600$; un numero davvero elevato per considerarsi un buon algoritmo per il clustering. Osservando i dati, in generale, si può notare come l'algoritmo Greedy con funzione di perdita VI e l'algoritmo Salso con la stessa portino a risultati molto simili tra loro, addirittura uguali per $N = 300$. La sostanziale differenza

si nota tuttavia nei tempi computazionali dove l'algoritmo Salso è computazionalmente meno oneroso. Basti osservare che per $N = 1600$ impiega poco più di un minuto per ottenere la partizione ottima, mentre l'algoritmo Greedy ne impiega almeno 49 (29467 secondi). Questo aspetto rilevante tende a farci preferire il Salso per l'applicazione successiva sui dati reali, nonostante a livello di metriche sia peggiore per N relativamente bassi dell'algoritmo Greedy con *VI*. Il miglioramento del Salso sostanzialmente avviene per N elevati, dato che per $N = 1600$, il numero di osservazioni "miss clusterizzate" dal Salso con *VI* risulta essere inferiore a quelle del greedy. Inoltre, le metriche $B(\hat{c}, c)$, $VI(\hat{c}, c)$, *ARI* evidenziano risultati migliori per l'algoritmo Salso, ricordando che $VI(\hat{c}, c)$ e $B(\hat{c}, c)$, essendo funzioni di perdita, portano a risultati migliori quando il loro valore è piccolo. In David B. Dahl and Müller (2022) è infatti fatto notare come questo algoritmo sia ottimo e più funzionale dei più semplici algoritmi greedy quando si tratta di clusterizzate strutture di dati complesse. Aumentando la numerosità campionaria abbiamo aumentato l'incertezza all'interno dei dati e dei loro pattern.

In questo studio di simulazioni si sono confrontati i due algoritmi: Greedy e Salso, con le rispettive loss function: *Binder* e *VI*, osservando il loro funzionamento al variare di alcuni parametri e al variare della numerosità campionaria. Si è arrivati alla conclusione che i due algoritmi, pur mostrando risultati molto simili tra loro, differenziano per tempi computazionali e per precisione al crescere di N . Riteniamo il Salso un algoritmo più efficiente per numerosità campionarie elevate, mentre il Greedy più efficace per numerosità ristrette e dati più semplici. Si sono, inoltre, implementate le credible balls sull'algoritmo Greedy, in modo da fornire una misura dell'incertezza della partizione ottima stimata, ottenendo risultati simili per le due funzioni di perdita. Un limite tuttavia di questo studio di simulazione è che i dati sono stati creati ad hoc per ottenere i risultati sperati, ossia le componenti del modello mistura sono state assunte ben separate tra loro. Dopo aver provato ad aggiungere rumore tra le componenti, i risultati ottenuti non rispecchiavano completamente le attese e gli algoritmi proposti hanno perso di efficacia. Si è arrivati alla conclusione che questi algoritmi sono ottimi in caso di cluster ben separati, e altrettanto efficienti nel fornire una stima dell'incertezza della partizione ottima ottenuta. Tuttavia quando i dati sono più complessi, non sempre la partizione ottima rispecchia i veri pattern da cui sono generati i dati.

4.2 Country HELP dataset

I modelli presentati nella Sezione 4.1, sono poi applicati al dataset *Country-Data*, reperibile su Kaggle e messo a disposizione da HELP International, una Onlus fondata a Torino nel 1995 che agisce con interventi umanitari a favore di popolazioni a rischio. Contiene varie misurazioni di quantità socio-economiche e di salute per 167 paesi per un totale di 9 variabili. Le osservazioni si riferiscono all'anno 2010. Il tema è aiutare HELP International a spendere i soldi delle donazioni nella maniera più strategicamente efficiente. L'Onlus deve decidere su quali nazioni direzionare il capitale e deve capire quali possano essere le popolazioni e i contesti che richiedono maggior bisogno di aiuto. Il lavoro che si deve svolgere è quindi quello di categorizzare le nazioni utilizzando fattori socio-economici e di salute per determinare il benessere di una popolazione o di una regione della Terra. In seguito, per compiere un ulteriore approfondimento, si è ritenuto opportuno fare un confronto con la situazione globale aggiornata. Si sono considerati dati messi a disposizione dalla World Bank Group sul loro sito Web. La World Bank Group è un'organizzazione che si concentra su cinque aree di sviluppo: Persone, Prosperità, Pianeta, Infrastrutture e Ambiente Digitale; lavorano con governi e settore privato fornendo aiuto tecnico e finanziario laddove è necessario, in particolare in zone turbate da guerre, violenza e disuguaglianza. I dati più recenti messi a disposizione, riferiti a 159 paesi, sono relativi all'anno 2022.

Per *Country-Data* si considera un modello di mistura di Processo di Dirichlet, più precisamente Diagonal Location Scale, come visto in Sezione 4.1. Anche in questo caso al parametro di massa è data una hyperprior Gamma(1,1) in accordo con Escobar and West (1995). I parametri del modello sono stati scelti in modo empirico, per $i = 1, \dots, p$, con $p = \#feature = 7$: $k0_i = 2$, $m0_i$ pari alla media campionaria della variabile i , $a0_i = 2$ e $b0$ pari alla diagonale della covarianza campionaria dei dati. Inoltre non sono state poste delle hyperprior sui parametri appena citati. Per fare inferenza, come nell'esempio di simulazione 4.1 si è usato un marginal Gibbs Sampler (Neal, 2000) con 10000 iterazioni e 1000 di burn-in. Per verificare che si arrivi a convergenza è stato utilizzato lo stesso metodo visto in

Sezione 4.1. Sia il traceplot del numero di cluster, che quello dell'entropia che il test di Geweke danno risultati che portano a concludere che l'algoritmo è arrivato a convergenza.

In Sezione 4.1 si sono confrontati diversi algoritmi e diverse funzioni di perdita al fine di trovare la combinazione che portasse alla soluzione più efficiente per fare inferenza a posteriori. Lo studio ha evidenziato come l'algoritmo Salso di David B. Dahl and Müller (2022) combinato con la loss VI (Meilă, 2007) sia la scelta migliore per il problema in questione pertanto ci siamo limitati all'utilizzo di quest'ultimo.

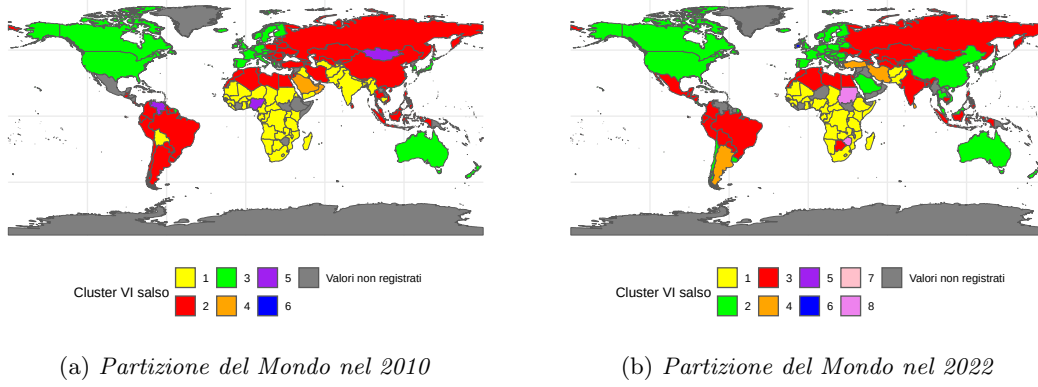


Figura 7: Il Mondo a distanza di 12 anni: risultati dell'applicazione di modelli mistura di processo di Dirichlet per fare clustering

Per quanto riguarda il dataset riferito al 2022, le variabili coinvolte nell'analisi sono due di meno rispetto a *Country-Data*, in particolare mancano *Health*, indice che misura la percentuale di spesa per la salute pro capite, e *Income* che rappresenta il reddito netto pro capite. Uno spunto per un approfondimento dello studio potrebbe essere proprio quello di capire quanto pesi la presenza (o l'assenza) di queste due variabili nella categorizzazione di uno Stato e quindi nella realizzazione del clustering.

Anche in questo caso si è scelto lo stesso modello di mistura di processo di Dirichlet, con la differenza che $k_0 = (1/2, \dots, 1/2)$, vettore di dimensione $p = 7$ e che il numero di iterazioni per il marginal Gibbs Sampler è pari a 20000 con 5000 di Burn-In. Utilizzando i metodi già esposti, non si riscontrano problemi che possano far pensare alla mancata convergenza dell'algoritmo.

In Figura 7 si possono notare le partizioni ottime individuate nei due casi. In Figura 7a viene mostrata la partizione ottima del Mondo nel 2010. Il DPMM distingue gli stati in 6 cluster diversi; è piuttosto immediato il riconoscere un pattern in questa suddivisione che di certo supporta la fiducia nei modelli di mistura di processo di Dirichlet come strumento per fare clustering. Possiamo infatti notare come nel cluster "Verde" vengano raggruppati tutti quegli Stati che possono essere ricondotti al concetto di Occidente, non geografico quanto politico ed economico, cioè quelle nazioni alleate degli USA con elevato livello di sviluppo del tessuto socio-economico. Pertanto vengono inclusi anche Giappone, Corea del Sud, Australia e Nuova Zelanda che si trovano in regioni orientali della Terra. Il cluster "Rosso", invece raggruppa tutti quei paesi che, almeno nel 2010, potevano essere considerati del Secondo Mondo, cioè quegli stati con livelli variabili di sviluppo economico, spesso con forte industrializzazione, ma con carenze nei beni di consumo e standard di vita non sempre elevati. Due esempi notevoli sono la Federazione Russa e il Brasile, caratterizzati da un tessuto socio-economico non particolarmente sviluppato, ma da una disponibilità di materie prime importanti, come gas naturale e legno, elevata. Fanno parte di questo cluster anche Marocco, Algeria, Libia ed Egitto, distinguendosi dai paesi del proprio continente che sono in maggioranza appartenenti al cluster "Giallo". Questo può essere dovuto alla loro vicinanza con i paesi europei, in maggioranza appartenenti al cluster "Verde", che li ha permesso di intrattenere scambi economici e culturali che li hanno portati ad essere gli stati più all'avanguardia dell'Africa. Si nota in particolar modo che il cluster "Giallo" è composto da nazioni situate nell'Africa subsahariana e nel subcontinente indiano. Si può ritenere che questo raggruppamento faccia quindi riferimento a quei

paesi etichettati come Terzo Mondo, cioè meno sviluppati dal punto di vista economico, politico e del benessere della popolazione.

E' interessante soffermarsi sul raggruppamento "*Arancione*", perchè come si vede in Figura 6, ne fanno parte solo sette stati: Bahrain, Brunei, Kuwait, Oman, Qatar, Arabia Saudita e Emirati Arabi. Sono tutti paesi della penisola arabica e principali esportatori nel mondo di petrolio. Sono per questo nazioni particolarmente ricche, ma di una ricchezza distribuita in modo molto diseguale, in mano di pochi appartenenti all'élite economica e politica del paese. Questa è, in modo abbastanza convincente, una delle principali ragioni che li distingue da quei paesi "*Verdi*". Da far notare anche che l'unica nazione della penisola araba a non appartenere a quel cluster è lo Yemen che, sebbene abbia riserve ingenti di petrolio, non era (e non è) un grande produttore rispetto agli altri paesi della regione; le motivazioni derivano dalle difficoltà interne, come la guerra civile, la corruzione e la scarsa infrastruttura. Nel raggruppamento "*Viola*" troviamo Guinea Equatoriale, Mongolia, Nigeria e Venezuela. Stati geograficamente molto lontani fra di loro, ma che nel 2010 avevano aspetti in comune, seppur con le loro differenze, come sfide politiche, dipendenza dalle risorse naturali e disuguaglianze sociali ed economiche. L'ultimo cluster individuato in Figura 7a è quello "*Blu*" non visibile sulla mappa. Le ragioni si riconducono al fatto che i 3 stati (da Tabella 6) in "*Blu*" in Figura 6 sono Lussemburgo, Malta e Singapore cioè paesi accomunati dalla loro dimensione piccola e dal regime fiscale favorevole verso imprenditori e investitori internazionali che li fa aggiudicare l'etichetta di paradiso fiscale. Interessante come ultima osservazione da fare per la partizione del 2010 è la diversa numerosità dei cluster individuati. Infatti risulta dalla Tabella 6 che i cluster più numerosi siano quello "*Giallo*" e quello "*Rosso*". Diversamente, per la partizione individuata con dati aggiornati al 2022, il cluster più numeroso risulta essere quello "*Verde*", ciò può essere un indice di come in un intervallo di tempo di dodici anni la qualità della vita globale sia migliorata.

	Giallo	Rosso	Verde	Arancione	Viola	Blu
Numerosità Cluster 2010	58	66	29	7	4	3

Tabella 6: Numerosità di ciascun Cluster individuato nel 2010

	Giallo	Rosso	Verde	Arancione	Viola	Blu	Rosa	Violetto
Numerosità Cluster 2022	41	47	58	5	1	4	1	2

Tabella 7: Numerosità di ciascun Cluster individuato nel 2022

Per questo secondo dataset la partizione ottima, come si vede in Figura 7b, si distingue in 8 cluster. I cluster "*Rosso*" e "*Giallo*" hanno la stessa interpretazione raccontata per il caso precedente, con alcuni cambiamenti notevoli che testimoniano il fatto di come la situazione globale non sia rimasta ferma nei dodici anni che separano i due grafici in Figura 7. Ad esempio possiamo notare come l'India fosse un paese "*Giallo*" nel 2010 e come invece sia diventato un paese definito come "*Rosso*" nel 2022. Questo cambiamento è in linea con quello che ci si poteva aspettare, essendo l'India un paese in forte crescita non solo dal punto di vista economico, ma anche politico e culturale. È ragionevole pensare di poter fare dei confronti tra cluster appartenenti alle due partizioni, come quello appena fatto, poiché si possono osservare dei pattern comuni tra la Figura 7a e la Figura 7b. Ad esempio il cluster "*Rosso*" condivide per la maggior parte in entrambi gli anni gli stessi paesi, che, come detto precedentemente, possono essere considerati del Secondo Mondo: nazioni con forte industrializzazione, con abbondanza di risorse naturali, ma con tessuto socio-economico e benessere generale della popolazione di scarso livello. Ragionamento simile può essere compiuto per il raggruppamento "*Giallo*" che contiene i paesi più poveri, con più disuguaglianze e in linea generale appartenenti a quella classe definibile come Terzo Mondo.

Convinti di queste motivazioni si approfondiscono la casistica di Cina, Arabia Saudita e Irlanda. Guardando alle Figure 8a e 8b possiamo notare come per la Cina vi sia un importante spostamento verso la coda di destra della densità. Questo è un risultato in linea con quanto si può vedere nella Figura 7b, perché ora la Cina, che nel 2010 era uno stato "Rosso", fa parte del cluster "Verde", che è quello che ospita i paesi più sviluppati del globo: quelli occidentali. Non solo, è anche un risultato in linea con il senso comune; sappiamo che nell'ultimo decennio questa nazione ha conosciuto uno sviluppo del tessuto socio-economico importante, nonché ha assunto una rilevanza politica centrale nello scacchiere internazionale. Un discorso simile per certi versi si può fare anche per l'Arabia Saudita, la quale è passata da essere un paese "Arancione" ad essere un paese "Verde". In questo caso le ragioni non vanno ricercate in un maggior sviluppo del reparto economico come possiamo dedurre dalle Figure 8a e 8b, ma quanto in un miglioramento della propria proposta culturale nel Mondo. Un esempio significativo può essere la Lega di Calcio Araba che negli ultimi anni è riuscita a riunire tanti calciatori importanti strappandoli anche a squadre europee. Un ultimo esempio di cambiamento lo si può notare con l'Irlanda, la quale è passata ad essere uno di quei paesi "Blu", che anche nella partizione del 2022 coincidono con i paradisi fiscali Lussemburgo, Malta e Singapore. Questa svolta dello stato è in linea con quello che possiamo vedere nelle due Figure 8a e 8b, anche qui si vede uno spostamento verso la coda destra della distribuzione.

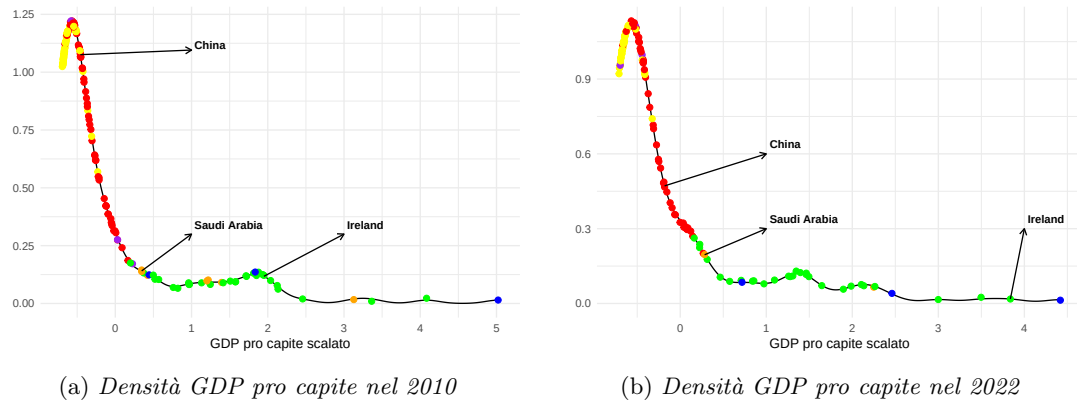


Figura 8: In (a) densità della variabile GDPP scalata nel 2010 con le osservazioni colorate secondo il cluster di appartenenza; In (b) densità della variabile GDPP scalata nel 2022 con le osservazioni colorate secondo il cluster di appartenenza nel 2010.

Come detto precedentemente la partizione del 2022 individua ben 8 cluster, 2 di più rispetto a quella del 2010. Nonostante ciò questi raggruppamenti aggiuntivi sono composti da pochissimi stati e non è facile darne un'interpretazione semplice e immediata.

Si propone infine come spunto di approfondimento la mappa in Figura 8c, pensata dalla rivista

italiana di geopolitica Limes e presente nel volume del Febbraio 2020. Questa mappa rappresenta la suddivisione per Limes del Mondo in due parti: Caoslandia e Ordolandia. La prima è caratterizzata da nazioni in conflitto o con situazioni di particolare instabilità, mentre la seconda coinvolge paesi con un certo grado di benessere della popolazione e un certo livello di stabilità garantita dalle istituzioni politiche. È interessante quindi notare come vi possano essere similitudini rispetto ciò che propongono gli studiosi della rivista con quello che si è trovato. Si può affermare che approssimativamente quello che viene considerato Caoslandia coincida con gli Stati appartenenti al cluster "*Giallo*" di entrambe le partizioni, quella del 2010 e del 2022. Al contrario il cluster "*Rosso*" e soprattutto quello "*Verde*" comprendono quelle nazioni che Limes definisce come stabili e quindi appartenenti ad Ordolandia. Alla luce di queste considerazioni ulteriori si può dire che i modelli di mistura di DP per fare clustering hanno portato a risultati sensati e per un futuro sviluppo si potrebbe approfondire la questione anche coinvolgendo direttamente esperti di altri campi come quello geopolitico o della sociologia.

Riferimenti bibliografici

- D. A. Binder. Bayesian cluster analysis. *Biometrika*, 65(1):31–38, 04 1978. ISSN 0006-3444. doi: 10.1093/biomet/65.1.31. URL <https://doi.org/10.1093/biomet/65.1.31>.
- Riccardo Corradin, Antonio Canale, and Bernardo Nipoti. Bnpx: An r package for bayesian nonparametric modeling via pitman-yor mixtures. *Journal of Statistical Software*, 100:1–33, 2021.
- D. B. Dahl. Model-based clustering for expression data via a dirichlet process mixture model. *Cambridge University Press*, page 201–218, 2006.
- D. B. Dahl and M. A. Newton. Multiple hypothesis testing by clustering treatment effects. *Journal of the American Statistical Association*, page 517–526, 2007.
- David B Dahl, Devin J Johnson, and Peter Müller. Search algorithms and loss functions for bayesian clustering. *Journal of Computational and Graphical Statistics*, 31(4):1189–1201, 2022.
- Devin J. Johnson David B. Dahl and Peter Müller. Search algorithms and loss functions for bayesian clustering. *Journal of Computational and Graphical Statistics*, 31(4):1189–1201, 2022. doi: 10.1080/10618600.2022.2069779.
- Michael D Escobar and Mike West. Bayesian density estimation and inference using mixtures. *Journal of the american statistical association*, 90(430):577–588, 1995.
- Thomas S Ferguson. A bayesian analysis of some nonparametric problems. *The annals of statistics*, pages 209–230, 1973.
- John Geweke. Getting it right: Joint distribution tests of posterior simulators. *Journal of the American Statistical Association*, 99(467):799–804, 2004.
- Marina Meilă. Comparing clusterings—an information based distance. *Journal of Multivariate Analysis*, 98(5):873–895, 2007. ISSN 0047-259X. doi: <https://doi.org/10.1016/j.jmva.2006.11.013>. URL <https://www.sciencedirect.com/science/article/pii/S0047259X06002016>.
- Radford M Neal. Markov chain sampling methods for dirichlet process mixture models. *Journal of computational and graphical statistics*, 9(2):249–265, 2000.
- Bernardo Nipoti and Weining Shen. Comment on ”bayesian cluster analysis: Point estimation and credible balls” by wade, s., and ghahramani, z. *Bayesian Analysis*, 13(2):579–584, 2018. doi: 10.1214/17-BA1073.
- Riccardo Rastelli and Nial Friel. Optimal bayesian estimators for latent variable cluster models. *Statistics and Computing*, 28:1169–1186, 2018.
- Jayaram Sethuraman. A constructive definition of dirichlet priors. *Statistica sinica*, pages 639–650, 1994.
- Sara Wade and Zoubin Ghahramani. Bayesian cluster analysis: Point estimation and credible balls (with discussion). 2018.